EAST WEST UNIVERSITY
Department of Computer Science and Engineering

CSE 475-Task2 Report

Course Name: Machine Learning

Course Code: CSE475

Section No: 06

Group No: 10

Dataset: Bot-IoT

Date of submission: 21th August 2026

Submitted To:

Dr. Raihan Ul Islam
Associate Professor,
Department of Computer Science & Engineering .

Submitted By:

Name: Habibur Rahman
ID: 2023-1-60-024

Name: Tamim Rafi Ahmed

ID: 2021-3-60-089

1. Introduction

Building on the exploratory data analysis carried out in Task 1, Task 2 moves from understanding the BoT-IoT dataset to actively modeling it. The objective of this task is to build and evaluate three predictive models on a focused binary classification problem — distinguishing HTTP flows from TCP flows — and to compare a purpose-built Graph Neural Network (GNN) against two well-established classical machine learning baselines: Logistic Regression and Random Forest. This comparison is central to the project's research question: does relational (graph-based) reasoning provide any measurable benefit over classical, feature-only models on a dataset that does not have a naturally occurring graph structure?

The HTTP-vs-TCP subcategory pair was deliberately selected (rather than the full multi-class attack-category problem explored in Task 1) because it represents a smaller, cleaner, two-class problem, letting the experiment isolate the effect of model architecture from the effect of severe class imbalance that dominates the full dataset.

2. Data Preparation

2.1 Loading and Filtering

The same auto-detecting loader used in Task 1 (load_dataset) was reused to bring in the 5% sample of the BoT-IoT dataset (four CSV files). From the full set of subcategories, only rows labeled "HTTP" or "TCP" in the "subcategory" column were retained, restricting the problem to a binary classification task. Duplicate rows were dropped and the index was reset.

2.2 Column Handling

The same identifier/leakage-prone columns removed in Task 1 were dropped again here: pkSeqID, stime, ltime, saddr, daddr, sport, dport, smac, dmac, soui, doui, sco, dco. Numeric columns with missing values were imputed with the column median; categorical columns were label-encoded using scikit-learn's LabelEncoder. The target column "subcategory" was separately label-encoded into the two classes {HTTP, TCP}.

2.3 Train/Test Split and Scaling

The dataset was split 80/20 into training and test sets using a fixed random seed (42) and stratified sampling to preserve the class ratio in both splits. Feature values were then standardized using StandardScaler (fit on the training set, applied to both sets), which is important both for the Logistic Regression baseline (a scale-sensitive linear model) and for the GNN (a gradient-based neural model).

Split	Shape (rows × features)
Training set	1,276,523 × 38

Test set	319,131 × 38

Classes present in the target: HTTP, TCP.

3. Baseline Models

Two classical, non-graph machine learning models were trained as baselines, both sourced from the scikit-learn library rather than implemented from scratch, so that the comparison against the custom GNN rests on well-understood, industry-standard reference points. A full discussion of exactly where these baselines come from and why they were chosen is provided in the companion document "Task2_Baseline_Model_Explanation.docx".

3.1 Logistic Regression

```
LogisticRegression(max_iter=1000, class_weight='balanced', random_state=42)
```

A linear classifier that models the log-odds of the target class as a linear combination of the standardized input features. class_weight='balanced' automatically re-weights the loss function in inverse proportion to class frequency, which protects against the class imbalance observed in Task 1.

3.2 Random Forest

```
RandomForestClassifier(n_estimators=100, class_weight='balanced', random_state=42,
n_jobs=-1)
```

An ensemble of 100 decision trees trained via bootstrap aggregation (bagging), with class-balanced weighting and parallel training across all available CPU cores.

3.3 Baseline Results

Model	Macro F1-Score	Accuracy	Training Time (s)
Logistic Regression	1.0000	1.0000	4.27
Random Forest	1.0000	1.0000	50.69

Both classical baselines achieve a perfect macro F1-score and accuracy of 1.0000 on the held-out test set. This striking result indicates that the HTTP-vs-TCP distinction is almost perfectly linearly separable from the available packet-level features (byte counts, packet counts, duration, rate, etc.) — a finding that is examined more closely through explainability (XAI) analysis in Task 3. Logistic Regression reaches this ceiling roughly 12 times faster than Random Forest (4.27s vs. 50.69s), making it the more efficient choice given equal predictive performance.

4. Graph Neural Network (GNN) Model

4.1 Motivation and Design Constraints

Unlike the tabular baselines, a GNN requires a graph structure — nodes connected by edges — over which it can perform message passing. The BoT-IoT flow records, however, do not carry an explicit graph: each row is an independent flow record with no encoded relationship to other

flows. To still test whether graph-style relational reasoning could help, each flow was treated as one graph node (its feature vector becoming the node feature), and a synthetic, randomly generated sparse edge set was constructed (function make_edges) connecting up to 4,000 random node pairs. This design choice is intentional and is revisited critically in the "Model Explanation" section below and in the ablation study performed in Task 3.

4.2 Architecture: SimpleGNN

The GNN was implemented from scratch in pure PyTorch (no external graph library such as PyTorch Geometric was used, ensuring it runs offline on Kaggle without additional dependencies). Its architecture consists of:

1. An input linear layer (in_channels → 128 hidden units) preceded by one round of mean-aggregation message passing, followed by BatchNorm and an ELU activation.
2. Dropout regularization (p = 0.3) to reduce overfitting.
3. A second linear layer (128 → 64 hidden units) preceded by a second round of message passing, again followed by BatchNorm and ELU.
4. A final linear output layer (64 → number of classes) producing class logits.

The message-passing step works by, for each node, averaging the feature vectors of its connected neighbors (using scatter_add for efficient aggregation) and adding that averaged neighbor signal to the node's own features before the next linear transformation — a residual-style combination of "self" and "neighborhood" information.

4.3 Training Configuration

Hyperparameter	Value
Hidden dimension	128 (then 64 in the second layer)
Dropout	0.3
Optimizer	Adam
Learning rate	0.001
Loss function	CrossEntropyLoss
Epochs	50
Random seed	42
Device	CPU (CUDA used automatically if available)

4.4 Training Curves

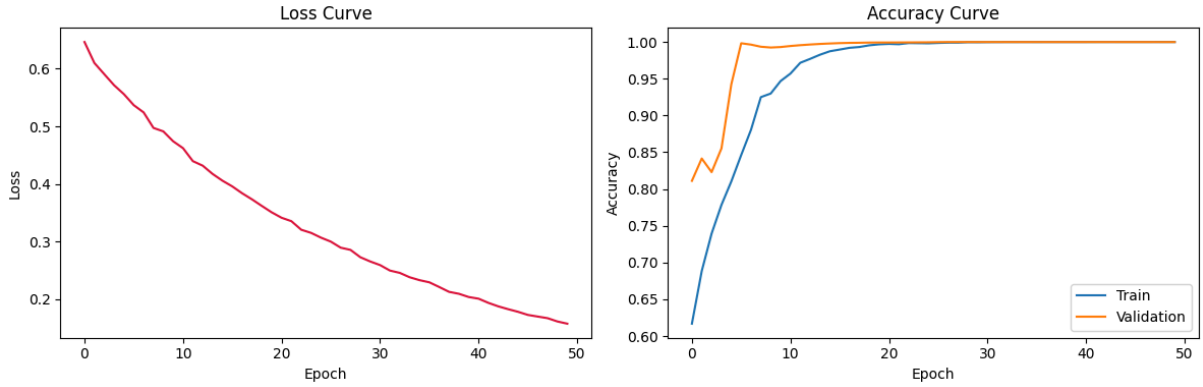

Figure 1. Left: training loss curve over 50 epochs, showing steady convergence. Right: training vs. validation accuracy curves, both rapidly approaching 1.0.

The loss curve decreases smoothly and monotonically across the 50 training epochs, indicating stable optimization without divergence. The accuracy curves for both the training and validation (test) sets rise quickly and converge close to each other near the top of the plot, showing no evidence of overfitting — the small gap between train and validation accuracy remains narrow throughout training.

4.5 GNN Results

Model	Macro F1-Score	Accuracy
SimpleGNN	0.9841	0.9999

The GNN achieves a very high macro F1-score of 0.9841 and an accuracy of 0.9999, but notably does not match the perfect 1.0000 macro F1 achieved by both classical baselines. This is an important and somewhat counter-intuitive result: the more architecturally complex, relationally-aware model performs slightly worse than the simplest possible linear classifier on this task.

5. Model Explanation (In the Team's Own Words)

The notebook includes a self-authored explanation of exactly how the SimpleGNN model processes data, reproduced and elaborated here:

5. Input: every network flow (one row of the dataset) becomes a single graph node; its features (packet counts, byte counts, duration, rate, etc.) become that node's feature vector.
6. Graph construction: since flows do not come with a natural graph in this dataset, nodes are connected using a random sparse edge set (make_edges). This gives the GNN some neighborhood to aggregate over, even though the edges themselves carry no real semantic meaning (they do not represent, for example, "these two flows share a source IP").

7. Message passing: each node averages its neighbors' features (mean aggregation) and adds that average to its own features, so its representation reflects both itself and its (randomly chosen) neighbors.
8. Linear + BatchNorm + ELU + Dropout: the combined signal is projected to a hidden dimension, normalized, passed through a non-linear activation, and regularized against overfitting via dropout.
9. A second message-passing and linear layer repeats step 3 for a "2-hop" view of the (random) neighborhood, and a final linear layer outputs class logits, which are converted to class probabilities via softmax during evaluation.

The team's own honest conclusion, stated directly in the notebook, is that: "this is an MLP with an extra 'look at your (random) neighbors' step bolted on — which is why it doesn't necessarily beat a plain classifier when the task itself needs no relational reasoning." This self-critical framing is an important part of the project's scientific honesty: rather than presenting the GNN uncritically as an improvement, the report acknowledges precisely why it may underperform, and sets up the ablation study carried out in Task 3 to test this hypothesis directly.

6. Final Comparison

Model	Macro F1-Score	Accuracy
Logistic Regression	1.0000	1.0000
Random Forest	1.0000	1.0000
SimpleGNN	0.9841	N/A (accuracy computed separately, 0.9999)

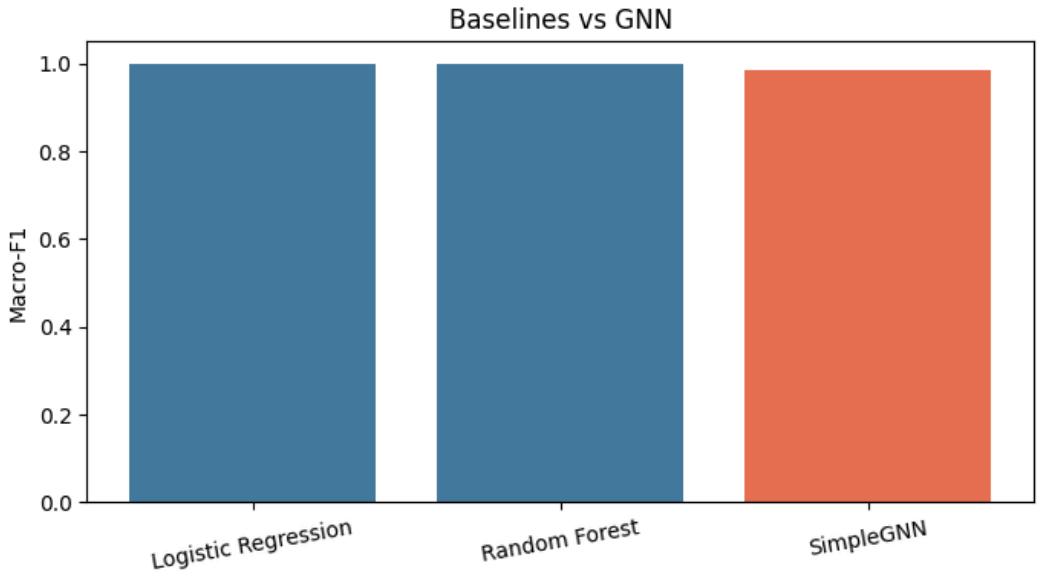

Figure 2. Bar chart comparing macro F1-score across the two baselines and the GNN. Best baseline: Logistic Regression, Macro-F1 = 1.0000; GNN Macro-F1 = 0.9841.

7. Discussion

The results of Task 2 reveal a clear and important pattern: on the HTTP-vs-TCP classification problem, the classical baselines (especially Logistic Regression) already reach a performance ceiling, achieving perfect scores in a fraction of the training time required by the neural GNN. This suggests that the underlying task is close to linearly separable given the available packet-level features, and that the GNN's additional architectural complexity — built around a synthetic, semantically meaningless edge structure — does not provide additional benefit and in fact introduces a small performance gap relative to the simplest baseline. This finding is consistent with the research gap identified in Task 1: published graph-model results on this dataset family tend to involve tasks or graph constructions with genuine relational structure (e.g., real host-to-host communication graphs), which is fundamentally different from the synthetic-edge setting explored here.

These observations motivate the deeper investigation carried out in Task 3, where an ablation study systematically removes or alters individual components of the GNN (including the message-passing step itself) to isolate exactly which architectural choices help or hurt performance, and where an explainability (XAI) analysis on the Logistic Regression model is used to identify precisely which features drive the classification decision.

8. Conclusion

Task 2 successfully implemented and evaluated three models — Logistic Regression, Random Forest, and a custom-built SimpleGNN — on the HTTP-vs-TCP binary classification problem derived from the BoT-IoT dataset. Both classical baselines reached perfect macro F1-scores, while the GNN, despite being trained successfully and stably (as shown by its smooth loss and accuracy curves), reached a slightly lower macro F1-score of 0.9841. The team's transparent interpretation is that the task does not require relational reasoning, and that the GNN's message-passing mechanism — operating over a randomly constructed, semantically arbitrary edge set — does not contribute meaningful additional signal beyond what the raw per-flow features already provide. This conclusion is investigated further and confirmed through a dedicated ablation study in Task 3.